

```

// Gemini API Service
// Gọi Gemini API với Character System Prompts

import { CharacterId } from './types';
import { CHARACTERS } from './characterData';

const GEMINI_API_KEY = import.meta.env.VITE_GEMINI_API_KEY;
const GEMINI_API_URL = 'https://generativelanguage.googleapis.com/v1beta/models/g

export interface ChatMessage {
  role: 'user' | 'assistant';
  content: string;
}

export interface CharacterChatResponse {
  characterId: CharacterId;
  characterName: string;
  message: string;
  affectionDelta: number;
  corruptionDelta: number;
}

export interface GroupChatMessage {
  characterId: CharacterId;
  characterName: string;
  message: string;
  timestamp: number;
}

/**
 * Call Gemini API for 1-on-1 character chat
 */
export async function callGeminiPrivateChat(
  characterId: CharacterId,
  playerMessage: string,
  chatHistory: ChatMessage[]
): Promise<CharacterChatResponse> {
  if (!GEMINI_API_KEY) {
    throw new Error('VITE_GEMINI_API_KEY not configured');
  }

  const character = CHARACTERS[characterId];
  if (!character) {
    throw new Error(`Character ${characterId} not found`);
  }

```

```

}

try {
  // Build conversation history for context
  const messages = [
    ...chatHistory.map((msg) => ({
      role: msg.role === 'user' ? 'user' : 'model',
      parts: [{ text: msg.content }],
    })),
    {
      role: 'user',
      parts: [{ text: playerMessage }],
    },
  ];

  const response = await fetch(GEMINI_API_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      contents: messages,
      systemInstruction: {
        parts: [
          {
            text: character.systemPrompt,
          },
        ],
      },
      generationConfig: {
        temperature: 0.85,
        topK: 40,
        topP: 0.95,
        maxOutputTokens: 1024,
      },
    }),
  });

  if (!response.ok) {
    const error = await response.json();
    throw new Error(`Gemini API error: ${error.error?.message || 'Unknown error'}`);
  }

  const data = await response.json();
  const characterResponse = data.candidates?.[0]?.content?.parts?.[0]?.text;

  if (!characterResponse) {

```

```

        throw new Error('No response from Gemini');
    }

    // Parse stats changes from response (subtle hints trong message)
    const affectionDelta = parseAffectionChange(characterResponse, characterId);
    const corruptionDelta = parseCorruptionChange(characterResponse, characterId)

    return {
        characterId,
        characterName: character.name,
        message: characterResponse,
        affectionDelta,
        corruptionDelta,
    };
} catch (error) {
    console.error('Gemini API error:', error);
    throw error;
}
}

/**
 * Call Gemini API for group chat (all F5 respond)
 */
export async function callGeminiGroupChat(
    playerMessage: string,
    chatHistory: ChatMessage[],
    selectedResponders: CharacterId[] = ['vincent', 'kaelen', 'dante', 'julian', 'e
): Promise<GroupChatMessage[]> {
    if (!GEMINI_API_KEY) {
        throw new Error('VITE_GEMINI_API_KEY not configured');
    }

    const responses: GroupChatMessage[] = [];

    try {
        for (const charId of selectedResponders) {
            const character = CHARACTERS[charId];
            if (!character) continue;

            // Build conversation history
            const messages = [
                ...chatHistory.map((msg) => ({
                    role: msg.role === 'user' ? 'user' : 'model',
                    parts: [{ text: msg.content }],
                })),
                {
                    role: 'user',

```

```

        parts: [{ text: playerMessage }],
    },
];

const response = await fetch(GEMINI_API_URL, {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
    },
    body: JSON.stringify({
        contents: messages,
        systemInstruction: {
            parts: [
                {
                    text: character.systemPrompt,
                },
            ],
        },
        generationConfig: {
            temperature: 0.85,
            topK: 40,
            topP: 0.95,
            maxOutputTokens: 512,
        },
    })),
});

if (!response.ok) {
    console.error(`Error getting response from ${character.name}`);
    continue;
}

const data = await response.json();
const characterResponse = data.candidates?.[0]?.content?.parts?.[0]?.text;

if (characterResponse) {
    responses.push({
        characterId: charId,
        characterName: character.name,
        message: characterResponse,
        timestamp: Date.now(),
    });
}

return responses;
} catch (error) {

```

```

        console.error('Gemini Group Chat error:', error);
        throw error;
    }
}

/**
 * Parse affection change từ character response
 * Dựa trên cách character react
 */
function parseAffectionChange(response: string, characterId: CharacterId): number {
    const lowerResponse = response.toLowerCase();

    // Positive affection signs
    if (
        lowerResponse.includes('em yêu') ||
        lowerResponse.includes('anh quan tâm') ||
        lowerResponse.includes('anh lo') ||
        lowerResponse.includes('em đặc biệt') ||
        lowerResponse.includes('anh cần em') ||
        lowerResponse.includes('em quan trọng')
    ) {
        return 5;
    }

    // Neutral
    if (
        lowerResponse.includes('anh') &&
        (lowerResponse.includes('sẽ') ||
            lowerResponse.includes('có thể') ||
            lowerResponse.includes('tính toán'))
    ) {
        return 2;
    }

    // Negative affection
    if (
        lowerResponse.includes('không xứng') ||
        lowerResponse.includes('dùng') ||
        lowerResponse.includes('buồn')
    ) {
        return -3;
    }

    return 0;
}

/**

```

```

* Parse corruption change từ character response
* Dựa trên manipulation/darkness intensity
*/
function parseCorruptionChange(response: string, characterId: CharacterId): number {
    const lowerResponse = response.toLowerCase();

    // High corruption signs - manipulation, possession, control
    if (
        lowerResponse.includes('của anh') ||
        lowerResponse.includes('chỉ anh') ||
        lowerResponse.includes('phải nghe lời') ||
        lowerResponse.includes('sở hữu') ||
        lowerResponse.includes('kiểm soát')
    ) {
        return 8;
    }

    // Medium corruption - subtle manipulation
    if (
        lowerResponse.includes('anh protect') ||
        lowerResponse.includes('em cần anh') ||
        lowerResponse.includes('anh hiểu') ||
        lowerResponse.includes('isolation') ||
        lowerResponse.includes('người khác')
    ) {
        return 4;
    }

    // Light corruption - general control/dominance
    if (
        lowerResponse.includes('anh') &&
        (lowerResponse.includes('decide') ||
        lowerResponse.includes('command') ||
        lowerResponse.includes('control'))
    ) {
        return 2;
    }

    return 0;
}

/**
* Get character info for display
*/
export function getCharacterInfo(characterId: CharacterId) {
    return CHARACTERS[characterId];
}

```

```

/**
 * Test API connection
 */
export async function testGeminiConnection(): Promise<boolean> {
  if (!GEMINI_API_KEY) {
    console.error('VITE_GEMINI_API_KEY not set');
    return false;
  }

  try {
    const response = await fetch(GEMINI_API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        contents: [
          {
            role: 'user',
            parts: [{ text: 'Hello' }],
          },
        ],
        generationConfig: {
          maxOutputTokens: 100,
        },
      })),
    );

    return response.ok;
  } catch (error) {
    console.error('Gemini connection test failed:', error);
    return false;
  }
}

```